

Logika

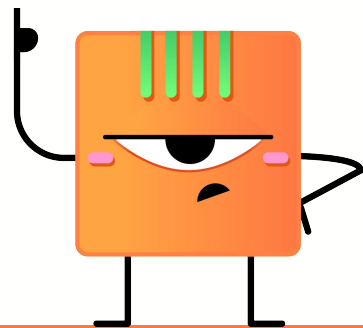
МОДУЛЬ 6. УРОК 9

# Шаблоны Jinja



## План уроку

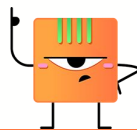
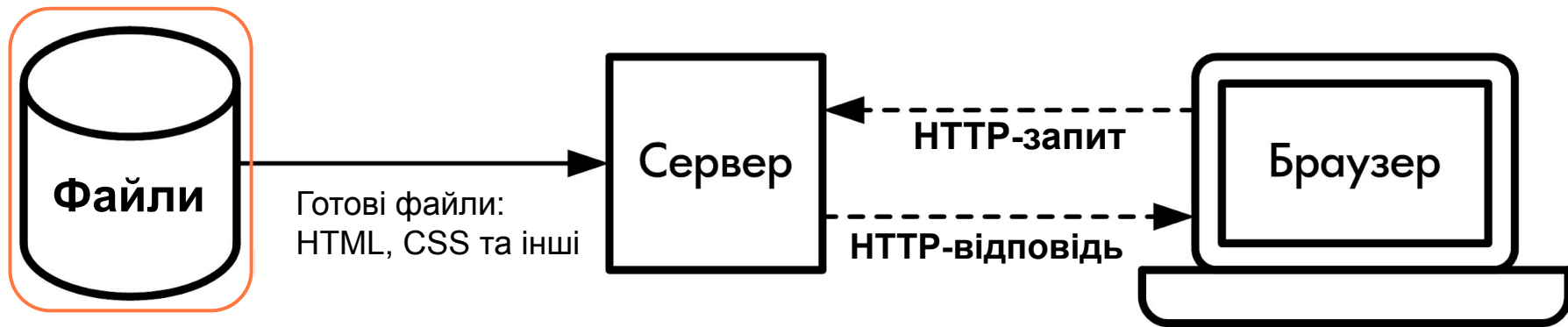
1. FlaskLogin
2. Шаблони
3. Додавання меню



# Схема роботи статичного сайту

**Сервер** — містить та передає інформацію;  
**Клієнт** — бажає отримати цю інформацію.

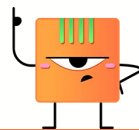
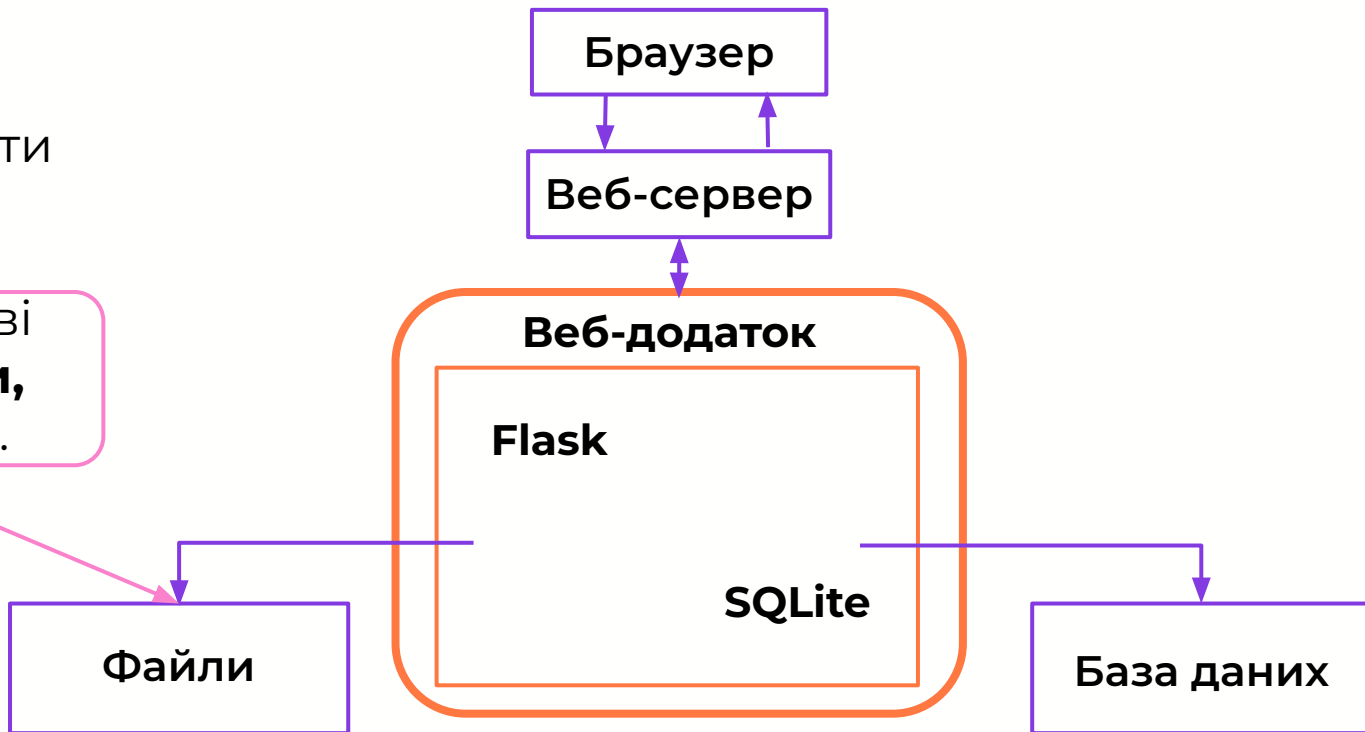
Готові  
файли: HTML,  
CSS та інші



# Файлова система

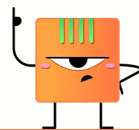
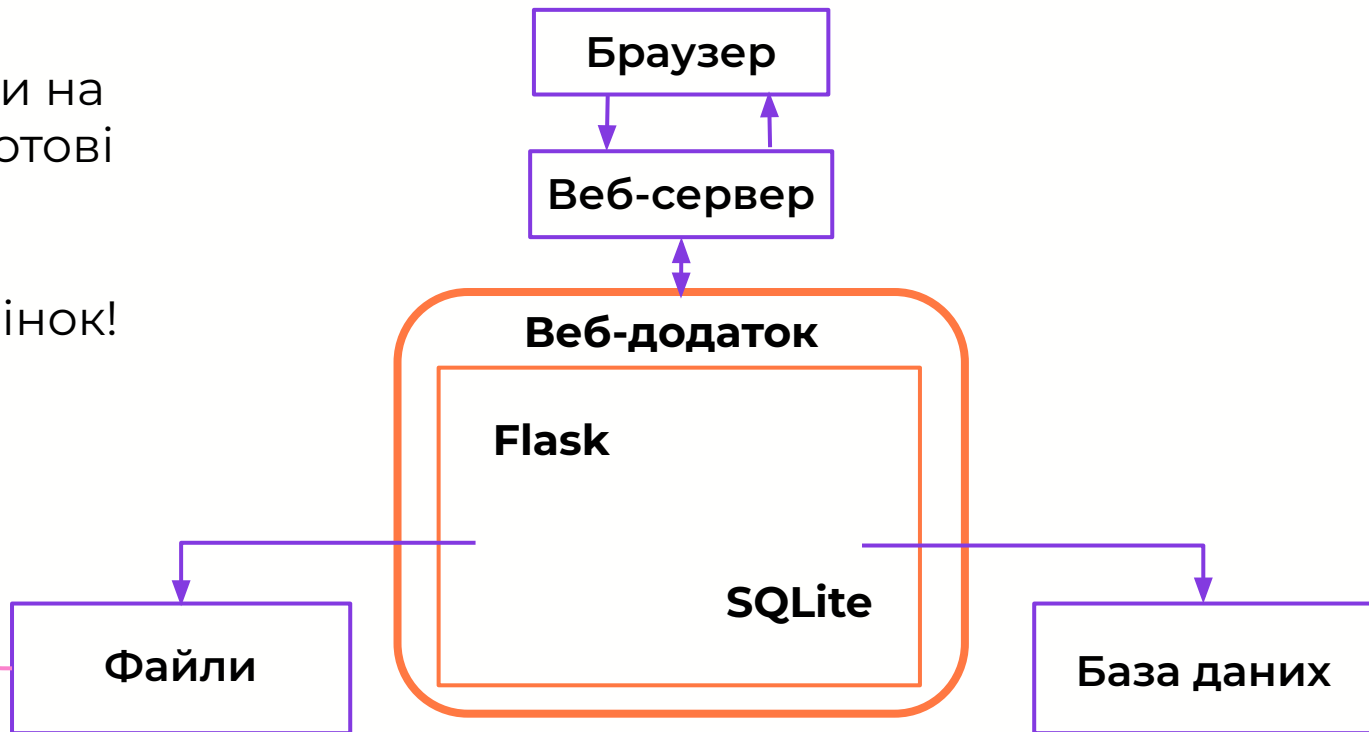
Flask може  
використовувати  
**статистичні**  
ресурси:

картинки, ГОТОВІ  
**HTML-сторінки,**  
таблиці стилів.



# Шаблони

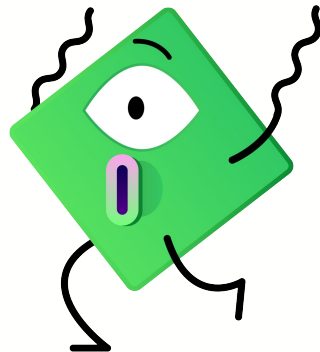
Можна покласти на диск не лише готові сторінки, а й **шаблони** для створення сторінок!





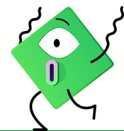
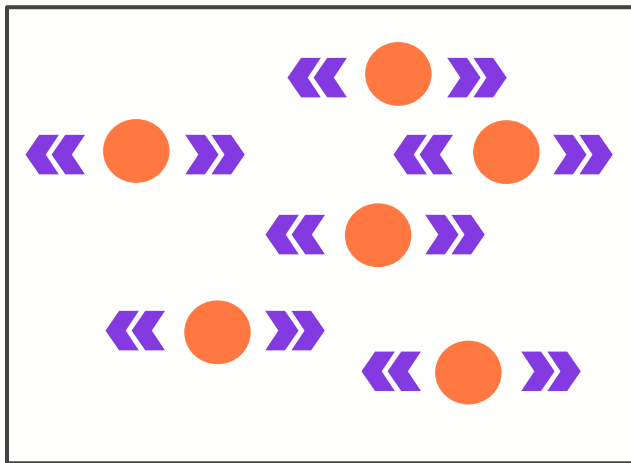
Модуль 6. Урок 7

# Обговорення: HTML-шаблони



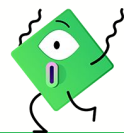
# Підхід «від даних до тегів»

Ми створювали HTML код навколо отриманих даних, обертаючи дані тегами.



# Недоліки програмування «від даних до тегів»

- Складно зібрати без помилок HTML-код.
- Важко змінювати код.
- Важко підтримувати однакове оформлення різних сторінок.
- Важко розподіляти роботу між учасниками проекту (дизайнером та backend-розробником).







МОДУЛЬ 6. УРОК 8

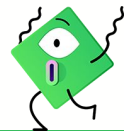
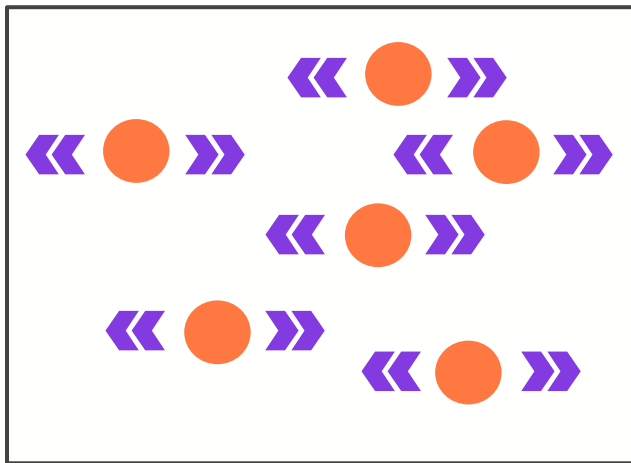
# Шаблоны



Скачать методичку

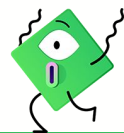
# Підхід «від даних до тегів»

Ми створювали HTML код навколо отриманих даних, обертаючи дані тегами.



# Недоліки програмування «Від даних до тегів»

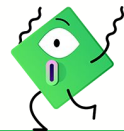
- Складно зібрати без помилок HTML-код.
- Важко змінювати код.
- Важко підтримувати однакове оформлення різних сторінок.
- Важко розподіляти роботу між учасниками проекту (дизайнером та backend-розробником).



# Шаблон

**Шаблон** — документ, оформлений за правилами HTML.

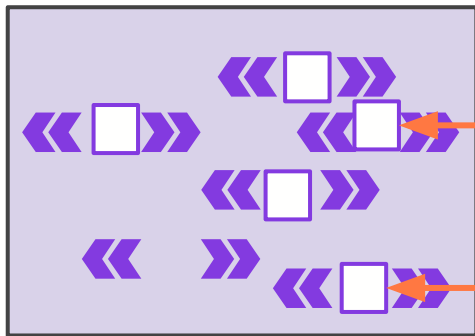
- Шаблон містить всю інформацію, що стосується **зовнішнього вигляду** сторінки.
- Шаблон не містить **конкретні дані**.
- У шаблоні вказано, **куда** які дані потрібно **вставляти**.



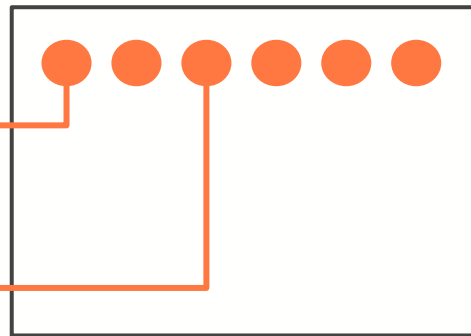
# Відокремлюємо оформлення від даних

Ми створюємо структуру HTML-коду з тегів, яку потім вписуємо потрібні дані.

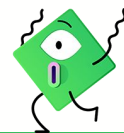
Теги



Дані

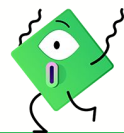


! Програма складається із ізольованих частин.



# Переваги шаблонів

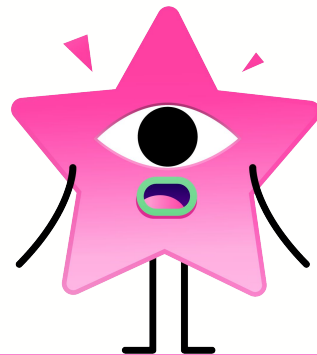
- Зручно збирати **HTML-код**.
- Легко вносити **зміни**.
- Не потрібно щоразу заново писати код, щоб **оформити** різні **сторінки однаково**.
- Над ізольованими частинами програми можуть працювати **різні фахівці**.





Модуль 6. Урок 7

# Нова тема: Обробка шаблонів



# Процес обробки шаблону

Обробляються **тільки** елементи у фігурних дужках {}, решта тексту повертається **як є**.

**{{Вираз}}** (у подвійних дужках) - це команда підставити в текст значення виразу. У виразі можуть бути використані передані параметри, операції з ними, виклики функцій.

```
<body>  
<h2>{{ header }}</h2>  
<p>{{ text }}</p>  
</body>
```





# Підстановка значень (4.py)

Значення параметрів для обчислення виразів повинні передаватися під час виклику **render\_template**:

```
render_template('index1.html',  
               header=" А якщо спробувати ", text="<i>Курсив?</i>")
```

Елементи оформлення розміщуються у шаблоні, дані **не повинні** містити таку інформацію.



# Вставка посилань значень (5.py, index.html)

Подвійні фігурні дужки:  
підставляється результат виклику  
функції

Правило "static" застосовується для  
створення посилань на статичні  
ресурси

```
href="{{ url_for('static', filename='style.css') }}"
```

Функція **url\_for** доступна процесу обробки  
шаблону без особливих вказівок  
(недоступні функції можна передавати  
параметрами)

Файл повинен знаходитись у  
папці, вказаній як static\_folder



# Це все?

Чи зможемо ми вирішувати наші завдання за допомогою такого процесу обробки шаблонів?

Як скласти шаблон для форми вибору вікторини?



# Управління процесом обробки

У процесі обробки шаблону можуть знадобитися **цикли та розгалуження** :

**{% оператор %}**

*(у фігурних дужках та відсотках) - команда управління процесом.*

Обробка шаблонів не залежить від відступів! Весь текст після оператора вважається **вкладеним** в нього, доки не зустрінеється **завершальний оператор**.



# Демо: умовний оператор

файли: [6.py](#)

[index3.html](#)

```
{% if header %}  ←  
  <h2>{{hello}}</h2>
```

```
{% else %}  ←  
  <h2>Немає заголовок</h2>
```

```
{% endif %}  ←
```

`{% if умова %}` — подальший текст буде виведено, якщо умова вірна

`{% else %}` — подальший текст буде виведено, якщо умова в `if` неправильно

`{% endif %}` — подальший текст не залежить від умови в `if`



# Демо: умовний оператор

Умови складаються так само, як на Python:

- оператори порівняння `==` , `!=` , `<` , `>` і т.д.;
- оператори **and**, **or**, **not**.

Змінна повертає **False**, якщо вона не має значення.



# Демо: цикл for



[скачать  
файли](#)

Використовуємо шаблон для сторінки вибору вікторини.

```
<select name="quiz">
```

```
{% for id, name in q_list %}
```

```
    <option value="{{id}}">{{name}}</option>
```

```
{% endfor %}
```

```
</select>
```

`{% for змінні in список %}` — цикл із  
почерговим використанням значень  
у списку

`{% endfor %}` — кінець циклу

Файли: [quiz.py](#), [first.html](#).

Нова тема: Обробка шаблонів



# Пам'ятка: оператори керування

Умовний оператор:

```
{% if умова%}  
{% else %}  
{% endif %}
```

Створення/зміна значення  
змінної:

```
{% set ім'я = значення %}
```

Цикл:

```
{% for змінні in список %}  
{% endfor %}
```

[Посилання на документацію.](#)

Нова тема: Обробка шаблонів





Шаблонізатор **Jinja**, який використовує Flask, підтримує спадкування.

**Спадкування шаблонів** - це дуже потужний функціонал, який дозволяє уникнути повторення одного й того ж у різних шаблонах.

Будемо використовувати спадкування шаблонів при створенні нашого проекту.

Flask за замовчуванням буде шукати файли шаблонів в каталозі **templates** коріня нашого проекту.

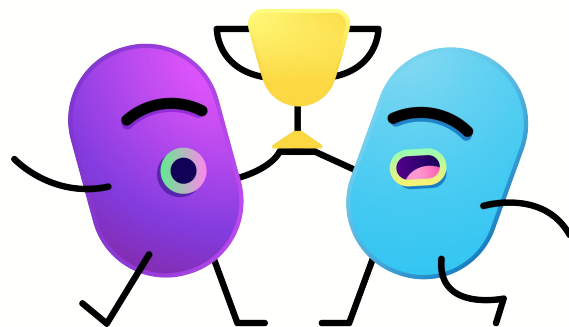
Почнемо зі створення файлу **index.html**





Модуль 6. Урок 5

# Завершения уроку



# Завершення уроку

Чого ви навчилися на цьому уроці?

Що найбільше сподобалося?

Що для вас було легко, а що складно?

Як ви могли б використати те, що дізналися на цьому уроці?

Які нові ідеї у вас виникли?



# Наступного разу...

...розглянемо процес  
отримання та обробки  
даних від користувача!

